

```
//bad way
var disableTask1 = false;
var defaultTask1 = 5;
var pointerTask1 = 2;
var disableTask2 = true;
var defaultTask2 = 10;
var currentValueTask2 = 10;
//like that many other variables
```

```
//better way
var task1 = {
  disable: false,
  default: 5,
  pointer: 2,
  getNewValue: function(){
    //do some thing
    return task1.default + 5;
  }
};
```

```
var task2 = {
  disable: true,
  default: 10,
  currentValue: 10
};
```

```
//how to use them
if(task1.disable){
  //do some thing...
  return task1.default;
}
```

7. *Use of callbacks:* When multiple functions are used in your code and if the second function is dependent on the effects of the first output, then callbacks are required to be written.

For example, task2 needs to be executed after completion of task1, or in other words, you need to halt execution of task2 until task1 is executed. I have noticed that many developers are not aware of callback functions. So, they either initialise one variable for checking [like mutex in the operating system] or set a timeout for execution. Below, I have explained how easily this can be implemented using callback.

```
//Javascript way

task1(function(){
  task1();
});

function task1(callback){
  //do something
```

```
if (callback && typeof (callback) === "function") {
  callback();
}
}
```

```
function task2(callback){
  //do something
  if (callback && typeof (callback) === "function") {
    callback();
  }
}
```

```
//Better jQuery way
$.task1 = function(){
  //do something
};
```

```
$.task2 = function(){
  //do something
};
```

```
var callbacks = $.Callbacks();
callbacks.add($.task1);
callbacks.add($.task2);
callbacks.fire();
```

8. *Use of 'each' for iteration:* The snippet below shows how each can be used for iteration.

```
var array;
//javascript way
var length = array.length;
for(var i =0; i<length; i++){
  var key = array[i].key;
  // like wise fetching other values.
}
```

```
//jQuery way
$.each(array, function(key, value){
  alert(key);
});
```

9. *Don't repeat code:* Never write any code again and again. If you find yourself doing so, halt your coding and read the eight points listed above, all over again. Next time I'll explain how to write more effective plugins, using some examples. 

By: Savan Koradia

The author works as a senior PHP Web developer at Multidots Solutions Pvt Ltd. He writes tutorials to help other developers to write better code. You can contact him at: savan.koradia@multidots.in; Skype: [savan.koradia](https://www.skype.com/en/contacts/savan.koradia)